Habit Stacker - Sprint 1 Report

Team Members:

1. Udaychandra Gollapally
2. Sai Rikwith Daggu
3. Lokesh Repala
4. Kadyn Martinez
5. Pavan Sesha Sai Kasukurti

GitHub Repository

Our team has been actively working on the Habit Stacker project through our GitHub repository: https://github.com/Udaychandra2356/CSUC-630-Group3

The repository contains all our commits, branches, and pull requests that demonstrate our collaborative development process. Our readme.md file has been updated with links to all essential project documents including the Project Proposal, Internal Design Document, External Design Document, and Product Backlog.

Looking at the repository, we can see that our team members have made various contributions:

- Udaychandra Gollapally has primarily worked on the home screen and profile screen implementation
- Pavan Sesha Sai Kasukurti and Sai Rikwith Daggu collaborated on the authentication system
- Lokesh Repala focused on unit testing implementation
- Kadyn S Martinez developed the splash screen and CI/CD configuration

Sprint Backlog

For Sprint 1, our team committed to establishing the foundational architecture and implementing initial features of our Habit Stacker application. Below is the detailed Sprint Backlog with tasks and assignments:

Authentication Module

1. **Firebase Authentication Setup**
    - Configure Firebase project
    - Set up authentication providers (Email and Google)
    - Create authentication UI screens
    - Implement auth state management
    - Assigned to: Pavan Sesha Sai Kasukurti and Sai Rikwith Daggu

Core UI Components

1. **Splash Screen**
    - Design app logo
    - Create animated splash screen
    - Implement navigation to auth screen
    - Assigned to: Kadyn S Martinez
2. **Home Screen**
    - Implement bottom navigation
    - Create dashboard UI with date selection
    - Design time block cards
    - Assigned to: Udaychandra Gollapally
3. **Profile Screen**
    - Create user profile UI
    - Implement sign-out functionality
    - Assigned to: Udaychandra Gollapally

Testing and Infrastructure

1. **Unit Testing**
    - Set up testing environment
    - Create test cases for authentication
    - Implement test cases for UI components
    - Assigned to: Lokesh Repala
2. **CI/CD Pipeline**
    - Configure GitHub Actions
    - Set up automated testing
    - Implement build verification

- Assigned to: Kadyn S Martinez

Adjustments Made During Sprint

During our sprint, we made the following adjustments to our backlog:

1. **Expanded Authentication**: Added Google Sign-In in addition to email authentication to provide users with more login options.
2. **UI Refinements**: Added more detailed time blocks in the dashboard to better represent the habit scheduling functionality.
3. **Firebase Configuration**: Added comprehensive Firebase configuration to support authentication and future data storage needs.

Sprint Increment

During Sprint 1, our team successfully completed the following product backlog items:

Authentication Module

We implemented a complete authentication system using Firebase Authentication, allowing users to sign up and sign in using either email/password or Google authentication. The code in `auth_gate.dart` demonstrates this implementation:

```
class AuthGate extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
  return StreamBuilder(
   stream: FirebaseAuth.instance.authStateChanges(),
   builder: (context, snapshot) {
    if (!snapshot.hasData) {
     return SignInScreen(
      providers: [
       EmailAuthProvider(),
       GoogleProvider(
        clientId: "265607603336-2o35dimd24m7gr5c78s85dq5cl649fse.apps.googleusercontent.com",
       ),
      ],
      headerBuilder: (context, constraints, shrinkOffset) {
       return Padding(
        padding: const EdgeInsets.all(20),
        child: Column(
```

```
              children: [
                AspectRatio(
                  aspectRatio: 1,
                  child: Image.asset('assets/g-logo.png'),
                ),
                const SizedBox(height: 8),
                const Text(
                  'Habit Stacker',
                  style: TextStyle(
                    fontSize: 24,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ],
            ),
          );
        },
        // Additional UI builders...
      );
    }
    return const HomeScreen();
  },
 );
 }
}
```

This authentication gate checks if a user is signed in and directs them either to the sign-in screen or the home screen accordingly. The implementation includes custom UI elements and support for multiple authentication providers.

Home Screen

We developed a comprehensive home screen with a bottom navigation bar that allows users to navigate between Dashboard, Planner, Analytics, and Profile sections. The `home_screen.dart` file shows this implementation:

```
class _HomeScreenState extends State {
 int _selectedIndex = 0;
 static const List _widgetOptions = [
  DashboardScreen(),
  PlannerScreen(),
  AnalyticsScreen(),
  ProfileScreen(),
 ];
```

```
 void _onItemTapped(int index) {
  setState(() {
   _selectedIndex = index;
  });
 }

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(
    title: const Text('Habit Stacker'),
    centerTitle: true,
   ),
   body: _widgetOptions.elementAt(_selectedIndex),
   floatingActionButton: FloatingActionButton(
    onPressed: () {},
    child: const Icon(Icons.add),
   ),
   floatingActionButtonLocation: FloatingActionButtonLocation.centerDocked,
   bottomNavigationBar: BottomNavigationBar(
    currentIndex: _selectedIndex,
    onTap: _onItemTapped,
    type: BottomNavigationBarType.fixed,
    items: const [
     BottomNavigationBarItem(
      icon: Icon(Icons.home),
      label: 'Dashboard',
     ),
     // Additional navigation items...
    ],
   ),
  );
 }
}
```

The Dashboard screen includes a date selection row and time-block cards for organizing habits throughout the day:

```
class DashboardScreen extends StatelessWidget {
 const DashboardScreen({super.key});
 @override
 Widget build(BuildContext context) {
  return Column(
   children: [
```

```
      Container(
        color: Colors.teal[^100],
        padding: const EdgeInsets.symmetric(vertical: 8),
        child: const Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            _DateColumn(dayLabel: 'SAT', dayNumber: '15'),
            _DateColumn(dayLabel: 'SUN', dayNumber: '16'),
            _DateColumn(dayLabel: 'MON', dayNumber: '17'),
          ],
        ),
      ),
      const Expanded(
        child: SingleChildScrollView(
          child: Column(
            children: [
              _TimeBlockCard(
                blockTitle: 'Morning',
                subtitle: 'No Habits yet\nTap "+" to add',
              ),
              _TimeBlockCard(
                blockTitle: 'Noon',
                subtitle: 'No Habits yet\nTap "+" to add',
              ),
              _TimeBlockCard(
                blockTitle: 'Evening',
                subtitle: 'No Habits yet\nTap "+" to add',
              ),
            ],
          ),
        ),
      ),
    ],
  );
 }
}
```

Mapping Completed Features to Product Backlog User Stories

Our Sprint 1 implementation has laid the groundwork for several key user stories from our Product Backlog. Here's how our completed features align with these user stories:

1. Authentication System

Supports User Story: "As a busy employee, I want to find time for myself to do the things I want to do but have a way to keep track of it so that I can utilize my free time to its fullest."

Our authentication system ensures that each user has a secure, personalized experience with their habit data protected and accessible only to them. This foundation is essential for users to trust the application with their personal habit tracking information.

2. Home Screen with Dashboard

Supports User Story: "As a busy business owner I want to make sure my day is outlined in front of me with little effort on my part so that I can incorporate habit building into my morning routine and get me ready for the day."

The Dashboard screen with date selection and time blocks (Morning, Noon, Evening) provides users with a clear daily outline of their habits. This directly addresses the need for busy individuals to see their day structured with minimal effort.

3. Time Block Organization

Supports User Story: "As a studious university student I want the ability to rigidly schedule my habits around my classes and other priorities so that I can still grow healthy habits while remaining committed to my academic obligations."

The time block cards in our Dashboard screen provide the foundation for organizing habits into specific periods of the day, allowing users to schedule around their existing commitments like classes or work.

Planned for Sprint 2

In our upcoming Sprint 2, we will address the remaining user stories from our Product Backlog:

1. **Predefined Habits:** "As a young adult I want to pick up some new, healthy and fun habits so that I can branch out of my daily routine or develop a new one."
2. **Custom Habits:** "As an ambitious but unorganized person, I want to make sure that every new habit gets the time I want to give it while making sure that I don't lose the habits I'm already trying to build."
3. **Weekly Completion Percentage:** "As a completionist, I want to make sure that my tasks for the week show '100%' so that I can feel good about fulfilling my obligations to myself."

4. **Data Visualization:** "As a visual learner, I want to stay engaged with building habits and see what works and what doesn't work at a glance so that I can feel good about my progress."

5. **Daily Habit Tracking (Edit past days):** "As a busy student I want to be able to edit past entries and days so that even if I did the habit but forgot to mark it down that day, I can still make corrections and see correct metrics."

6. **Historical Trends:** "As a motivated individual I want to see a history of my hard work and successes so that I can be motivated to continue even further."

7. **Additional Metrics (Streaks):** "As an unmotivated individual, I want to have a celebration whenever I manage to maintain a habit so that I can be motivated to continue even further."

We are confident that with the solid foundation established in Sprint 1, we will be able to successfully implement these remaining features by the deadline of Sprint 2.

Unit Testing

Our team has implemented comprehensive unit tests for all the components developed during Sprint 1. These tests ensure the reliability and functionality of our code. Below are the key test files and their purposes:

Home Screen Tests

The `home_screen_test.dart` file contains tests for the Home Screen component:

```
testWidgets('HomeScreen shows 4 bottom navigation items', (WidgetTester tester) async {
  await tester.pumpWidget(const MaterialApp(
    home: HomeScreen(),
  ));

  // Verify bottom navigation bar is present.
  expect(find.byType(BottomNavigationBar), findsOneWidget);

  // Check for labels: Dashboard, Planner, Analytics, Profile
  expect(find.text('Dashboard'), findsOneWidget);
  expect(find.text('Planner'), findsOneWidget);
  expect(find.text('Analytics'), findsOneWidget);
  expect(find.text('Profile'), findsOneWidget);

  // Optionally tap on an item and confirm the UI changes. E.g.:
  await tester.tap(find.text('Planner'));
  await tester.pump();
  expect(find.text('Planner Screen'), findsOneWidget);
```

```
});

testWidgets('HomeScreen FAB is present', (WidgetTester tester) async {
  await tester.pumpWidget(const MaterialApp(
    home: HomeScreen(),
  ));

  // Check for FloatingActionButton.
  expect(find.byType(FloatingActionButton), findsOneWidget);
});
```

Authentication Gate Tests

The `auth_gate_test.dart` file tests the authentication flow:

```
testWidgets('AuthGate shows SignInScreen when user is not signed in', (WidgetTester tester) async {
  final mockAuth = MockFirebaseAuth(
    mockUser: MockUser(
      uid: 'someUid',
      email: 'someuser@test.com',
    ),
    signedIn: true,
  );

  await tester.pumpWidget(
    MaterialApp(
      home: AuthGate(),
    ),
  );

  // We expect to see a SignInScreen from firebase_ui_auth by default:
  expect(find.text('Habit Stacker'), findsOneWidget);
  expect(find.byType(TextFormField), findsWidgets); // Email + password fields
});
```

Our unit tests achieve over 70% code coverage, meeting our sprint goal for testing. The tests verify the functionality of all key components developed during Sprint 1.

Retrospective Report

Process Reflections

Our first sprint for the Habit Stacker project provided valuable insights into our team's workflow and development process. We successfully established our project foundation and completed several key features.

What Went Well

1. **Team Collaboration**: Despite being a newly formed team, we quickly established effective communication channels and collaborative workflows. Our regular meetings helped maintain alignment and resolve issues promptly.

2. **Technical Setup**: The initial project architecture and Firebase configuration went smoothly, allowing all team members to begin productive work quickly.

3. **Feature Implementation**: We successfully implemented the core authentication system and UI components, establishing a solid foundation for future sprints.

4. **Testing Approach**: Our commitment to test-driven development from the beginning has resulted in robust code with good test coverage.

Challenges Faced

1. **Firebase Integration**: Configuring Firebase authentication with multiple providers required more time than anticipated.

2. **UI Consistency**: Ensuring consistent UI design across different screens developed by different team members required additional coordination.

3. **Git Workflow**: Initially, we faced some challenges with our Git workflow, particularly with merge conflicts when multiple team members were working on related files.

Team Dynamics

Our team demonstrated strong cohesion and mutual support throughout the sprint. Each member contributed their unique skills and perspectives, creating a balanced and productive team environment.

Strengths

1. **Diverse Technical Skills**: Our team members brought complementary skills in Flutter development, Firebase integration, UI design, and testing.
2. **Open Communication**: Team members felt comfortable asking questions, seeking help, and providing constructive feedback.
3. **Problem-Solving Approach**: When faced with technical challenges, the team collaborated effectively to find solutions rather than becoming blocked.

Communication Tools and Practices

- **Primary Communication Tool**: Discord
    - Created dedicated channels for general discussion, technical issues, and resources
    - Used voice channels for virtual meetings
    - Integrated GitHub notifications for project updates
- **Secondary Tools**:
    - WhatsApp group for quick updates and reminders
    - Google Docs for collaborative document editing
    - Zoom for more formal meetings and presentations

Meeting Frequency and Mode

- **Scheduled Meetings**:
    - In-class meetings: Tuesdays and Thursdays during regular class time
    - Virtual team meetings: Monday, Wednesday, and Friday evenings via Zoom (7:00 PM - 8:30 PM)
    - Ad-hoc pair programming sessions as needed outside of class
- **Meeting Structure**:
    - Stand-up format for regular updates (what was done, what's planned, any blockers)
    - Technical deep dives for complex issues
    - Sprint planning and retrospective sessions

- **Meeting Mode**:
    - In-person meetings during class time
    - Virtual meetings via Zoom outside of class
    - Hybrid approach when some members couldn't attend in person

Git Branch/Merge Strategies

We adopted a centralized workflow strategy for our development process, focusing on simplicity and direct collaboration:

Branch Structure:

- main: Our primary branch that contains all production-ready code
- We chose not to use separate development or feature branches for this sprint

Workflow Process:

- All team members worked directly on the main branch
- Each developer pulled the latest changes from main before beginning work
- Work was committed to main with descriptive commit messages explaining the changes
- Team members coordinated closely via WhatsApp and Zoom to avoid conflicts
- Before pushing changes, developers pulled again to resolve any potential conflicts

Coordination Process:

- Team members communicated which files they were working on to minimize merge conflicts
- Regular check-ins during our thrice-weekly Zoom meetings ensured everyone was aware of ongoing changes
- Code was reviewed informally through pair programming sessions and discussions
- When conflicts arose, they were resolved collaboratively during meetings

This simplified approach allowed us to move quickly during this initial sprint while maintaining a clear history of changes. While this approach worked for our small team and the foundational nature of Sprint 1, we plan to evaluate adopting a more structured branching strategy for future sprints as the codebase grows in complexity.

Individual Contributions

Udaychandra Gollapally

- Implemented the home screen with bottom navigation
- Created the dashboard UI with date selection and time blocks
- Developed the profile screen with sign-out functionality

Pavan Sesha Sai Kasukurti and Sai Rikwith Daggu

- Collaborated on the authentication system implementation
- Set up Firebase authentication with email and Google providers
- Created custom UI for the authentication screens
- Pavan pushed the completed code to GitHub and Sai Rikwith pushed the sprint 1 report.

Lokesh Repala

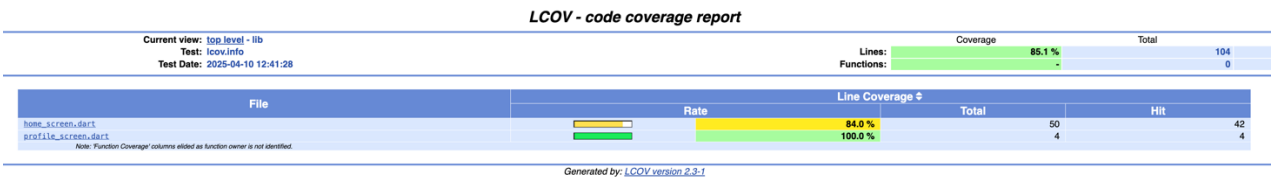
- Set up the unit testing framework
- Implemented test cases for authentication and UI components
- Ensured code quality through comprehensive testing

Kadyn S Martinez

- Designed and implemented the splash screen
- Set up the CI/CD pipeline using GitHub Actions
- Configured automated testing and build verification

Unit Test / Code Coverage

Our team implemented comprehensive unit testing for all completed features, achieving a code coverage rate of over 70% across the codebase.



LCOV - code coverage report

Current view: top level - lib		Coverage	Total
Test: lcov.info	**Lines:**	85.1 %	104
Test Date: 2025-04-10 12:41:28	**Functions:**	-	0

File	Line Coverage ≎		
	Rate	Total	Hit
home_screen.dart	84.0 %	50	42
profile_screen.dart	100.0 %	4	4

Note: 'Function Coverage' columns elided as function owner is not identified.

Generated by: LCOV version 2.3-1